

Report: Browser Extension to Block Trackers

Introduction

Online privacy is a critical concern as pervasive tracking scripts gather vast user data. This report details the development of "Privacy Shield," a browser extension designed to block known tracking scripts, empowering users to control their digital footprint.

Abstract

"Privacy Shield" is a privacy-focused browser extension for Chrome/Firefox, built on Manifest V3. It intercepts and blocks requests to known tracking domains using the [declarativeNetRequest](#) API. Key features include a real-time badge counter for blocked scripts and a user-friendly options page for managing custom whitelist and blacklist rules. The extension provides an efficient, user-configurable solution to enhance online privacy by preventing unwanted surveillance.

Tools Used

The "Privacy Shield" extension was developed using standard web technologies and browser-specific APIs:

- [JavaScript](#): Core language for all extension logic, including background service workers and UI interactions.
- [Manifest V3](#): The latest browser extension platform, emphasizing security, performance, and declarative approaches for network handling.
- [HTML](#): Structures the user interfaces for the extension's popup and options pages.
- [CSS](#): Styles the HTML elements for a clean and intuitive user experience.
- [Browser APIs](#):
 - [chrome.declarativeNetRequest](#): Essential for high-performance, privacy-preserving blocking of network requests based on static and dynamic rules.
 - [chrome.webRequest](#): Used heuristically for counting potential blocked requests.
 - [chrome.storage](#): Persists user-defined whitelist and blacklist rules.
 - [chrome.action](#): Manages the extension icon, including badge text for blocked script counts.
 - [chrome.runtime](#): Facilitates communication between different extension components.
 - [chrome.tabs](#): Allows interaction with browser tabs, such as reloading.

Steps Involved in Building the Project

Followed a structured development process, from configuration to UI design:

1. Project Setup and Manifest V3 Configuration:
 - An [manifest.json](#) file defined the extension's metadata, service worker (background.js), popup (popup.html), and crucial Manifest V3 permissions like declarativeNetRequest, storage, and host_permissions set to <all_urls>.
 - The [declarative_net_request](#) section was configured to load static blocking rules from [rules.json](#).
2. Tracker List Management:
 - A [rules.json](#) file was created to store static blocking rules. Each rule specifies an id, priority, action (e.g., block), and condition (including urlFilter for the tracking domain and resourceTypes).
 - The design noted the potential for integrating comprehensive third-party tracker lists like DuckDuckGo's Tracker Radar for broader coverage.
3. Intercepting and Blocking Requests (Background Service Worker - background.js):
 - The [background.js](#) service worker manages the core logic. It initializes a blockedCount to update the extension badge via updateBadge(), resetting on new page loads.

- A `chrome.webRequest.onBeforeRequest` listener increments `blockedCount` for URLs matching known tracker domains; `declarativeNetRequest` performs the actual blocking.
 - `addDynamicRule()` and `removeDynamicRule()` functions manage user-defined whitelist/blacklist entries by interacting with `chrome.declarativeNetRequest.updateDynamicRules()`. Dynamic rules are prioritized for flexibility.
 - `chrome.runtime.onMessage.addListener` enables communication for count retrieval and rule management from popup/options pages.
4. User Interface Development (Popup and Options Page):
- **Popup (`popup.html` and `js/popup.js`):** Displays the "Trackers Blocked on this Page" count, with buttons to "Manage Settings" and "Whitelist Current Site." It fetches data from the background script and uses a custom `alertMessage` for feedback.
 - **Options Page (`options.html` and `js/options.js`):** Provides a comprehensive interface for rule management, including input fields for domains, rule type selection, and lists for current whitelist and blacklist entries. It renders dynamic rules retrieved from the background script and enables adding/removing rules.
5. Essential Assets and Deployment:
- Placeholder icons were created in an icons directory. CSS and JavaScript files were organized into `css` and `js` directories, respectively.
 - Steps performed for loading the unpacked extension in developer mode in Chrome/Firefox for testing:-
 1. Open Chrome/Firefox: Go to `chrome://extensions` (for Chrome) or `about:debugging#/runtime/this-firefox` (for Firefox).
 2. Enable Developer Mode: Toggle "Developer mode" on (usually in the top right).
 3. Load Unpacked: Click "Load unpacked" (Chrome) or "Load Temporary Add-on" (Firefox).
 4. Select Directory: Navigate to your project folder (where `manifest.json` is located) and select it.

Conclusion

The "Privacy Shield" extension offers an effective solution for online privacy by blocking tracking scripts. Its use of Manifest V3's `declarativeNetRequest` API ensures efficient, browser-native blocking, while its user-friendly interface with real-time analytics and customizable rules empowers users. This project lays a strong foundation for a valuable tool in improving the browsing experience and securing personal data.